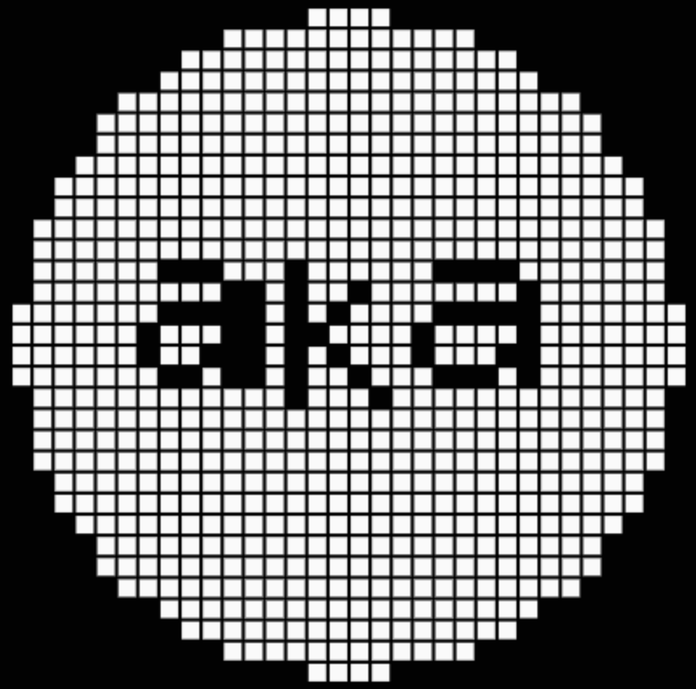


Ieuan King

PRODUCT DESIGN ENGINEER · TRAINED AS AN ANTHROPOLOGIST · BROOKLYN, NY

Portfolio, 2026

akabuild.dev ieuan@ubik.studio github.com/akaieuan

CONTENTS

Everything on this site is the same habit at different sizes: go and find the problem, build something you can actually operate, and keep the person using it in charge of what happens next.

Every page of this document is rendered by the same components, tokens and data as akabuild.dev, and printed from it. The write-ups it condenses are at the site, in full.

akabuild.dev ieuan@ubik.studio github.com/akaieuan

ABOUT · BROOKLYN, NY

How I work

Ieuan King. I'm a product design engineer trained as an anthropologist, which is mostly a way of saying I go and find the problem before I design for it. Most of that has gone into human-in-the-loop AI for expert users: sitting with the people doing the work, watching where the task actually breaks, and deciding what an agent should and should not be trusted with.

Field work, applied to product

It teaches you that what people say they do and what they do are different data, that both are worth collecting, and that the gap between them is usually where the product is wrong. Watching someone work around your software is worth more than any number of them rating it.

Designing through interaction

On a synth you do not describe the sound you want. You turn something and listen, and you keep turning until the room agrees with you. Software has the same property and most design process throws it away.

Small software

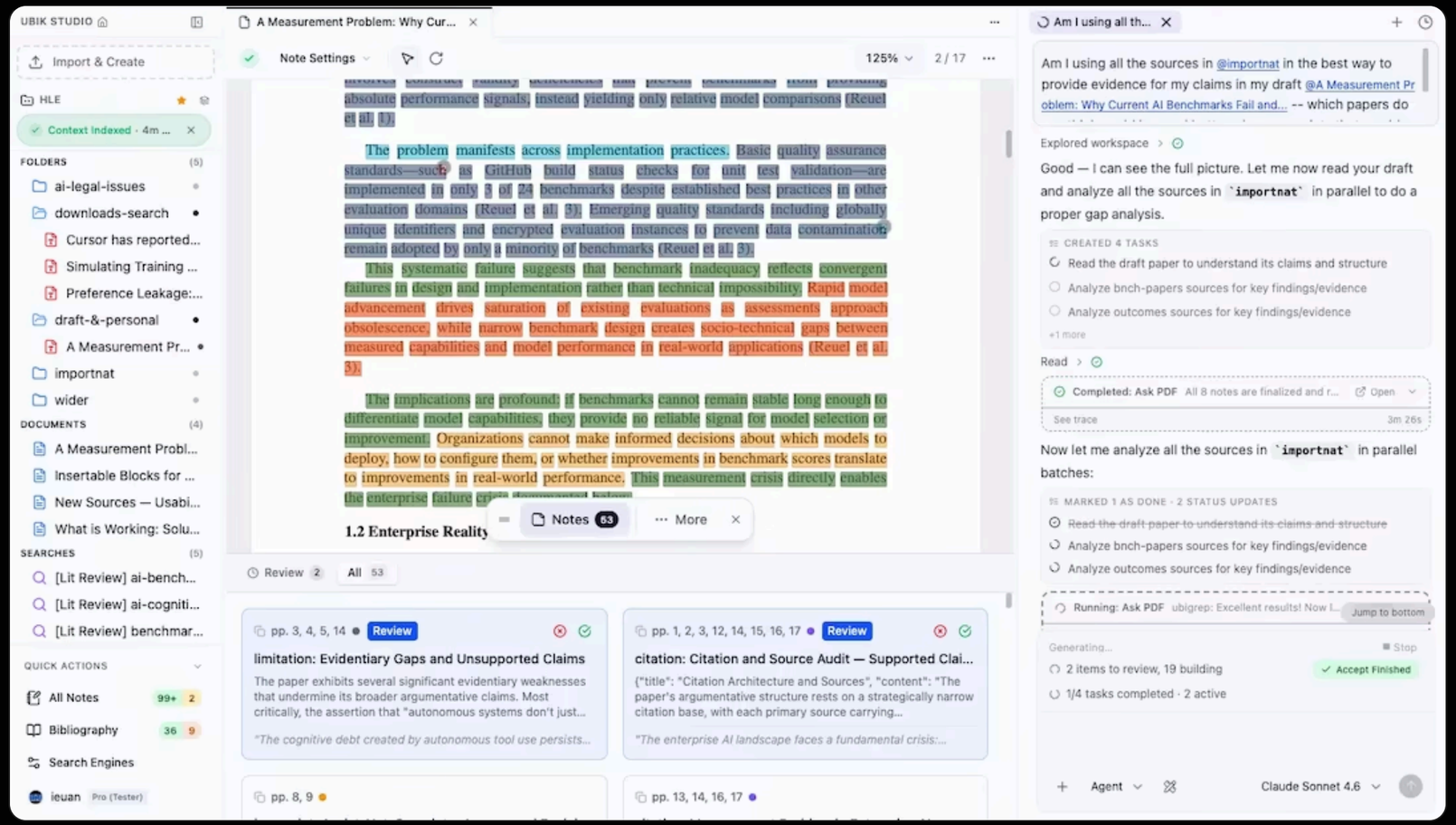
I believe in small software: a tool built for one person doing one thing, which is allowed to be opinionated precisely because it does not have to be for everyone.

PRODUCT · DESKTOP AI RESEARCH PLATFORM · 2023–2026

Ubik Studio

A desktop-native, local-first AI research platform. Three and a half years building the human side of agentic research, before it had a name.

2023–2026 · co-founded · the public site and builds are retired; the test log and the subreddit are what remain in the open.



The workspace in its final build: the file explorer and an indexed-four-minutes-ago context pill on the left, a source paper in the middle with every claim the agent drew highlighted in place, evidence cards queued for review along the bottom, and the agent working through a four-task plan on the right

In simple terms

Ubik Studio was a desktop program that worked like a research assistant: the AI did the reading and the drafting, and a person stayed in charge of every judgment call.

You pointed it at an ordinary folder of PDFs and papers on your own computer. It read and indexed them locally, searched across dozens at once, and pulled out what mattered. The rule it enforced was simple: the AI could not state a fact or write a sentence unless it could attach a direct quote and an exact page number from your own files.

When something needed a human decision, the agent stopped mid-task and said so. You got a review queue and approved, rejected or edited each claim before it reached the draft.

The impact. Most AI writing tools of the era tried to replace the researcher and shipped a wall of text nobody could check. Ubik did the opposite: it took the tedious half of the work and made the person verifying it the point of the product, which is the pattern the industry has spent the years since rebuilding.

› Why this was ahead of its time in 2023

Three and a half years

2023 to 2026, co-founded

1,038 commits

September 2023 to May 2026

Four surfaces

a desktop app, a web gateway, cloud agent deployments, and a browser extension

Local-first

the file system is the workspace

FROM THE WRITING AGENT'S SYSTEM PROMPT

> *“Your job is not to replace human thinking — it is to amplify it. Optimize for the loop: you draft, the human refines, you incorporate, the human approves. Intelligence is maximized not when either side works alone, but when the handoff between AI and human is so seamless it feels like one mind thinking.”*

That sentence was written years before “human-in-the-loop” became an industry talking point. Ubik spent three and a half years trying to actually earn it.

UBIK STUDIO

Why this was ahead of its time in 2023

In 2023 the state of the art in public was a single chatbot in a text box. It hallucinated freely, could not cite anything reliably, and left you copying snippets back and forth by hand. Ubik was already doing five things the field would spend the next several years arriving at.

Multiple specialised agents, not one general chatbot

Then you talked to one model that tried to do every part of the job at once.

Ubik split the work across sub-agents, a PDF reader, a literature researcher and a drafting agent, and let you assign a different model and reasoning depth to each. A cheap fast model ingested text; an expensive one wrote.

Citations enforced in code, not requested in a prompt

Then early chat-with-your-PDF tools were notorious for inventing quotes and giving page references that did not exist.

Ubik made it a programmatic rule. No exact quote and page number meant the claim could not be written at all, and every drafted line stayed linked to the highlighted source.

Human-in-the-loop as architecture rather than a confirmation dialog

Then you got either full automation or a tedious one-prompt-at-a-time conversation.

Ubik had a dedicated review queue and a Human Needed status. The agent paused at judgment calls, queued the evidence, and waited to be approved, rejected or corrected in batches.

Local-first, on your own machine

Then nearly everything required uploading confidential research to somebody else's cloud.

Ubik ran on the desktop against your real file system. Pointing it at an existing folder indexed it in place, with no proprietary silo to move your data into.

Reading many long documents at once

Then context windows were often four to eight thousand tokens, which made cross-referencing several papers essentially impossible.

Ubik let you mention a dozen papers in one prompt and read them in parallel against a working draft instead of summarising them one at a time.

While most companies that year were putting a wrapper around a chatbot, this was already the interaction design, the verification safeguards and the multi-agent workflow the rest of the field would go on to build.

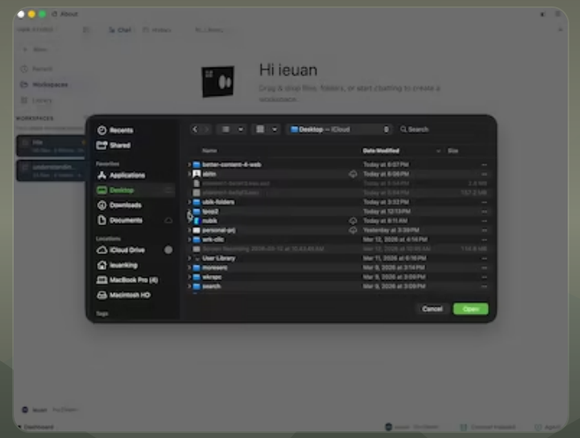
THE PRODUCT, IN MOTION

Seven silent recordings of the last build, March 2026.

A folder becomes a workspace

No import step and no database: point Ubik at a directory of PDFs and it indexes them in place, into a local context engine that reports how stale it is in the status bar. The explorer is the file system, so the workspace is still just a folder when you close the app.

1:25



Twelve papers into one prompt

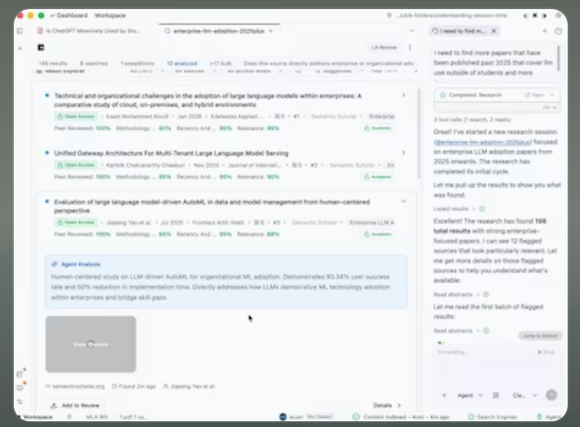
Sources are @-mentioned rather than uploaded. A dozen papers named in a single sentence, and the agent reads them in parallel against the draft rather than summarising them one at a time. The model picker sits beside the send button because which model reads your sources is a decision worth leaving with the researcher.

1:15

Search that scores its own results

Agentic literature search across 146 results and 8 searches, then the Result Explorer: every candidate scored on peer review, methodology, recency and relevance, with an agent analysis under each and a preview before anything is accepted into the workspace.

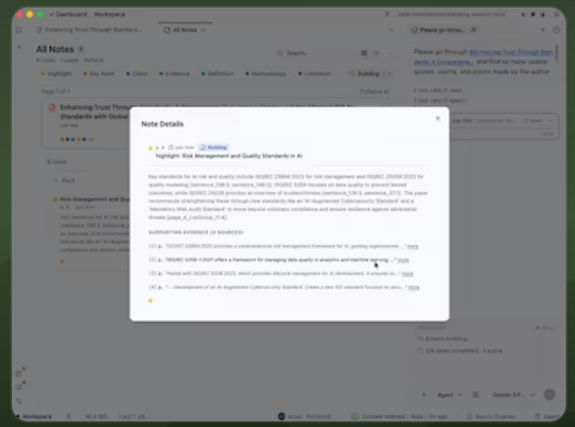
0:53



A note is its evidence

Notes are typed — highlight, key point, claim, evidence, definition, methodology, limitation — and opening one shows the supporting quotes with page numbers behind it. There is no note in Ubik that is only an assertion; if the quotes are not there, the note does not get written.

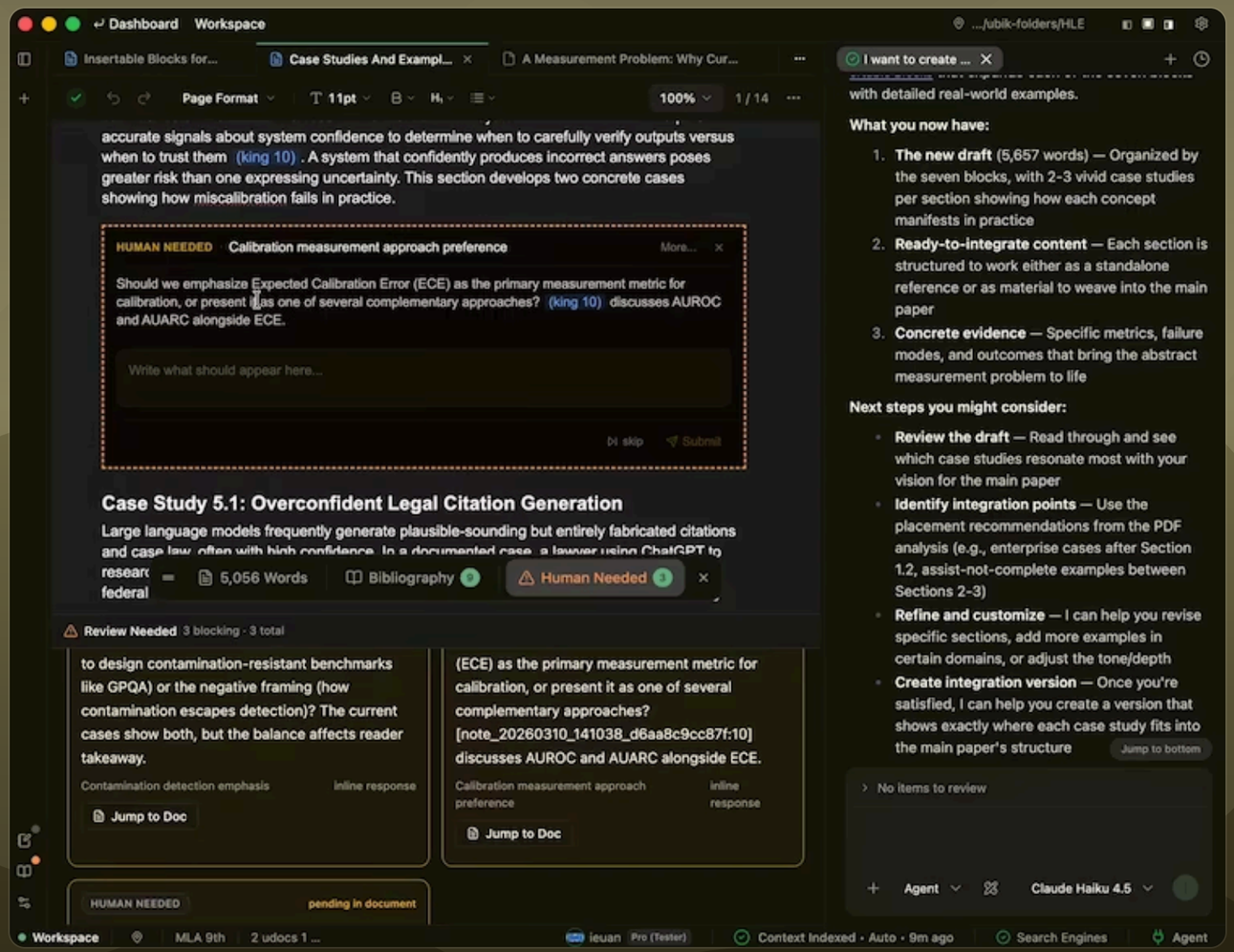
1:16



THE PRODUCT, IN MOTION

Human Needed, and the review queue

The part I am proudest of. The agent stops mid-document, states the judgment it needs, and waits — skip or submit, counted in the status bar as blocking. Beside it the review queue holds every claim the run produced, each one accepted or rejected on its own, with a jump straight to where it would land.



MY ROLE

I co-founded Ubik and led product design end to end: the workspace model, the review surfaces, the evidence and citation UX, the Human Needed grammar, and the copy and interaction conventions across every surface. I built front-end throughout, and ran the user research cycles — interviews, behavioral observation, session replays, and the team test log that documented them in public.

On the agent side I owned the system prompts, skills, and custom datasets — and I designed and built the **custom evaluation framework and ARC eval suite** we used to train, tune, and regression-test our agents and the agent-orchestration systems that coordinated them. That evaluation work is what actually moved the product: measurable gains in output accuracy, answer quality, and real-world usability — not benchmark numbers in isolation, but whether a researcher could trust and use what came back. It's the part of Ubik least visible in a screenshot and the part that mattered most to the results.

THE ENGINEERING

A large multi-package system: a desktop app, a web gateway, cloud agent deployments, and a browser extension, with a local-first storage model underneath it all. 1,038 commits from September 2023 to May 2026 — with the design and research that preceded the first commit, about three and a half years of my life.

`Electron · Next.js · TypeScript · Python · local-first`

A CLOSED CHAPTER

The public site and the builds are retired, and I'm at peace with that — Ubik was a complete thing, and it stands on its own. Three and a half years of asking one question in earnest: what does it take to make an AI research tool a person can actually trust?

DESIGN SYSTEM · LIVE SPECIMEN

akaSTYLE

The vocabulary every project on this site is built from: the tokens, the one type scale, the primitives, and the canvas engine that draws every brand mark. It exists as rules rather than as taste, and as something that renders itself rather than a document about itself.

In simple terms

The design language every project here is built from, written as nine rules instead of preferences.

A preference has to be re-argued every time. A rule can be checked in review and travels to a new codebase without me in the room.

The impact. It is why these projects look like one studio made them, and why an AI assistant can build a new surface in the same language without being taught it again.

THE MARK FAMILY

akaBuild
`wordmark void`

akaOSS
`sparkle void`

Games
`work line`

Not a logo file. A disc of cells with the wordmark subtracted, drawn at render time by the same engine every other mark in the family uses.

WHERE IT RUNS

akabuild.dev

This site. The faces hero, pixel aka wordmark, the project-card vocabulary, every write-up page.

`akabuild.dev`

akaoss.dev

The open-source studio site: same type scale and card system, with the sparkle mark as its brand variant.

`www.akaoss.dev`

HITL Kit

Nineteen human-in-the-loop React primitives, installable via the shadcn CLI. The interaction half of this language.

`akabuild.dev/demo/hitl-kit`

Trickle UI Kit

47 pure-CSS text-animation primitives: the motion rules above, packaged as zero-runtime components.

`akabuild.dev/demo/trickle-ui-kit`

boxpopuli.live · akacovart.com

Client and studio work that borrows the dark ground, bordered cards, and mono labelling wholesale.

`akabuild.dev/demo/box-populi`

THE RULES

Nine constraints, not nine preferences

01 Mono for structure, sans for prose.

Uppercase mono kickers at 11px/0.18em tracking label every section. Body copy is font-light sans with generous leading. The contrast between the two carries the hierarchy, so headings rarely need to be big. Sizes come from one closed scale, eight pixel steps and a display size, and nothing off it.

02 One accent, used sparingly.

Everything is greyscale on a near-black (or near-white) ground except a single quiet green. If two things on screen are competing for the accent, neither gets it. The five-hue accent set belongs to the art layer, not the interface; the one place it reaches the chrome is where hue is the value rather than decoration on one, as in the deck's position pill.

03 Light falls, nothing casts.

Depth is a 1px edge and a graded fill, never a drop shadow. A raised card grades light-to-dark downward and lifts its top edge; a recessed well inverts both. Fine grain over the fill keeps a gradient this shallow from banding. Rounded to 0.75rem for cards, 0.5rem for controls.

04 Motion moves space, never brightness.

Animation translates, scales, and displaces. It does not flash, strobe, or pulse opacity. This started as an accessibility rule for the audio-reactive work and became the house style.

05 Loops pause when unwatched.

Every canvas engine gates its RAF loop on an IntersectionObserver plus visibilitychange, and renders one still frame under prefers-reduced-motion. Ambient animation should cost nothing when nobody is looking.

06 Layout never jumps.

Tabbed regions are floored to the tallest tab at each breakpoint. Images ship with intrinsic dimensions and blur placeholders. Switching views should never move the content under a reader.

07 Server by default.

Components stay server-rendered unless they need state, an event, or a canvas. The client boundary is drawn as deep in the tree as possible: a card is a server component even when its page is interactive.

08 Per-frame state lives in the DOM.

Anything changing at sixty frames a second is written straight to a CSS variable or a data attribute and React is never told; React state is for what changes at human speed. The projects deck scrolls eighteen covers for zero re-renders, because its position is one custom property the CSS reads and every cover derives its own pose from it.

09 Every ink clears its ground.

Text is never mixed below the point where it reads at 4.5:1 on the page in both themes: the foreground stops at 60 percent, the muted ink and ink on art at 75. The quiet register is that floor, and hierarchy under it is size, case and tracking, which law 01 already gives. The check computes the floors from the tokens, and the sweep reads every page back through axe to prove them.

OPEN SOURCE V0.6

HITLKit

A design system, component library, and perspective paper on human-in-the-loop AI.
`hitlkit.dev`

In simple terms

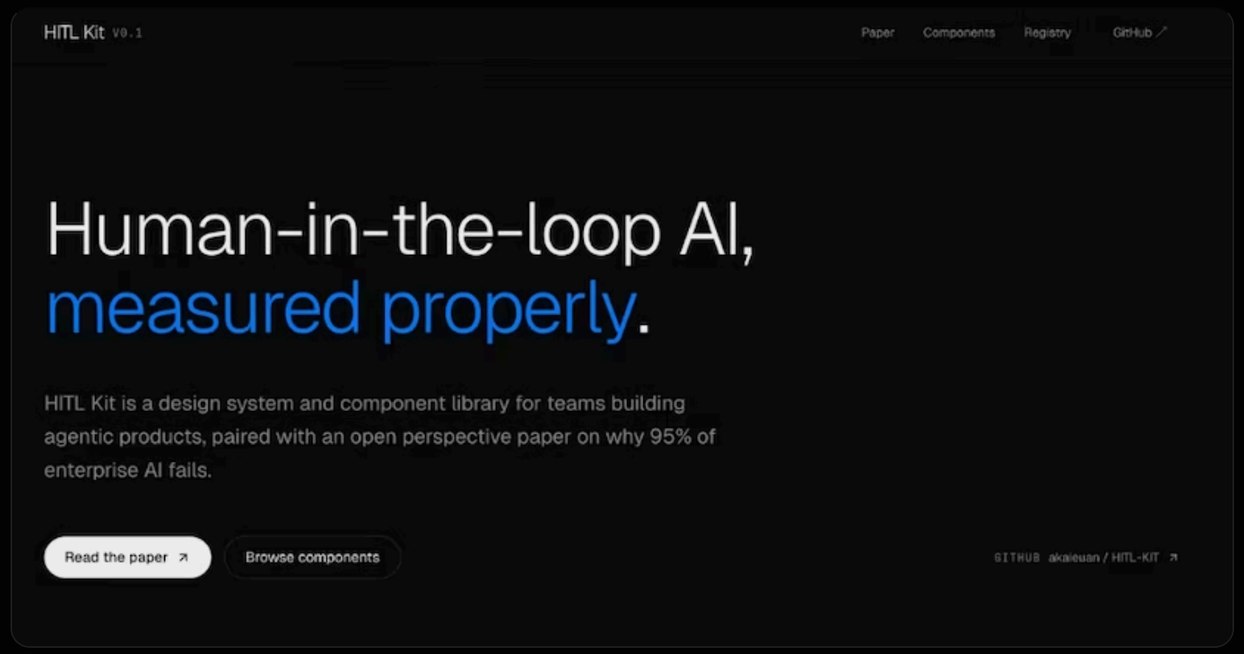
A free toolkit of nineteen ready-made interface pieces for products where an AI does work and a person has to sign off on it: approval rows, confidence meters, a preview of the action an agent is about to take, the evidence behind a claim it made.

Any developer installs one into their own project with a single command, with six npm packages behind them carrying the logic.

It ships with a paper. The argument is that most enterprise AI pilots fail because the industry tests the wrong thing: benchmarks ask whether a model can finish a task alone, while deployment asks whether it respected the authority of the person using it and left them better off.

Every primitive traces back to a specific claim in that paper. If a piece cannot be tied to one, it does not ship.

The impact. It closes the distance between an argument about how AI should work and a team being able to build it that way, which is otherwise months of design work a smaller team never gets to do.



Live site, paper, registry, and component showcase — the canonical home for the project.

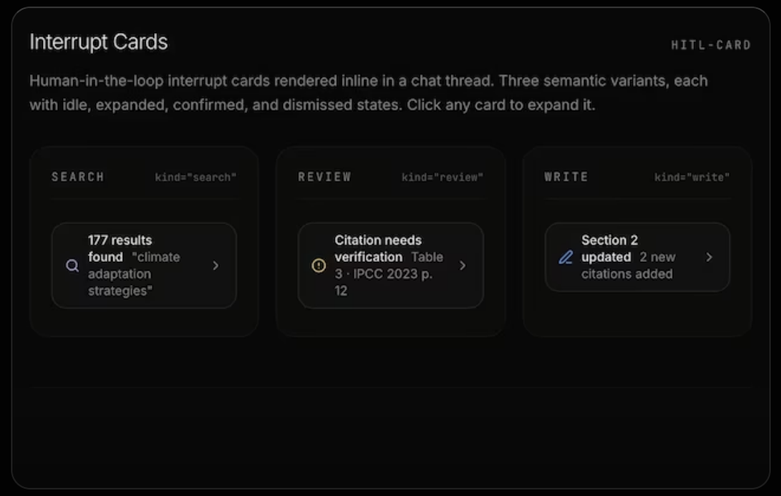
INSTALL ANY PRIMITIVE

```
npx shadcn@latest add https://www.hitlkit.dev/r/<id>.json
```

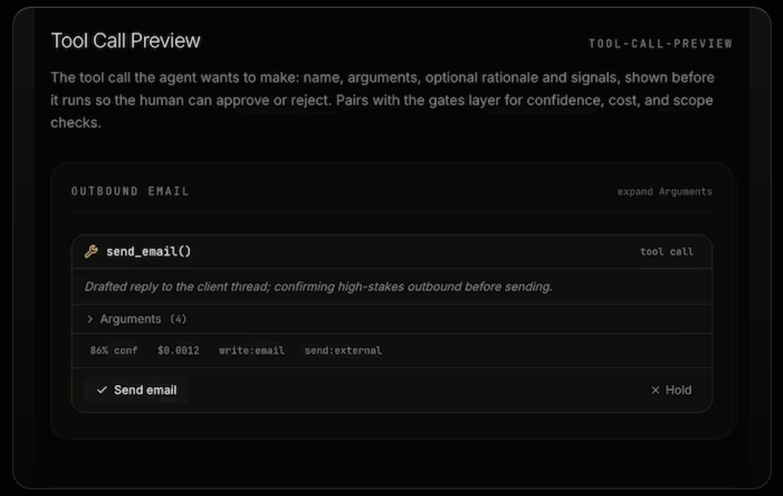
`github.com/akaieuan/HITL-KIT`

THE PRIMITIVES

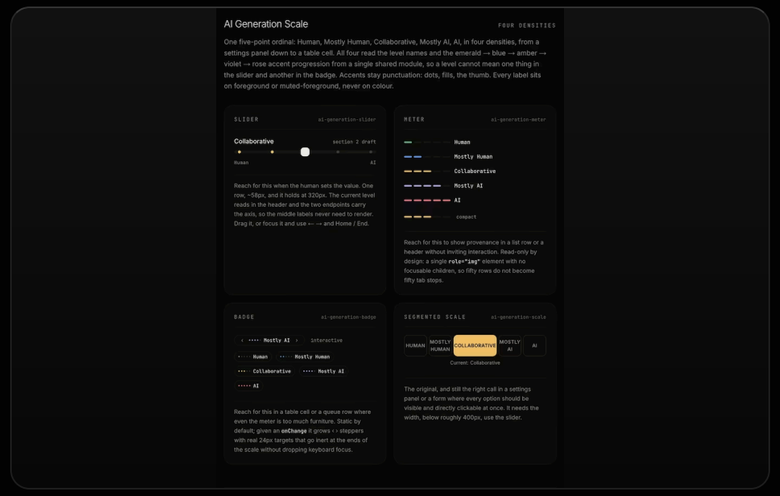
Six of the nineteen, as they render in the shipped library. Each one carries its registry id in the corner, so the picture and the install command name the same thing.



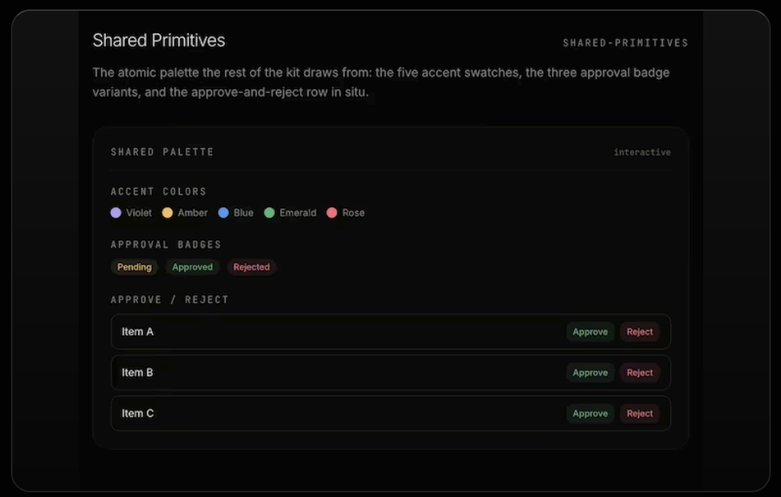
Interrupt Cards `hitl-card`



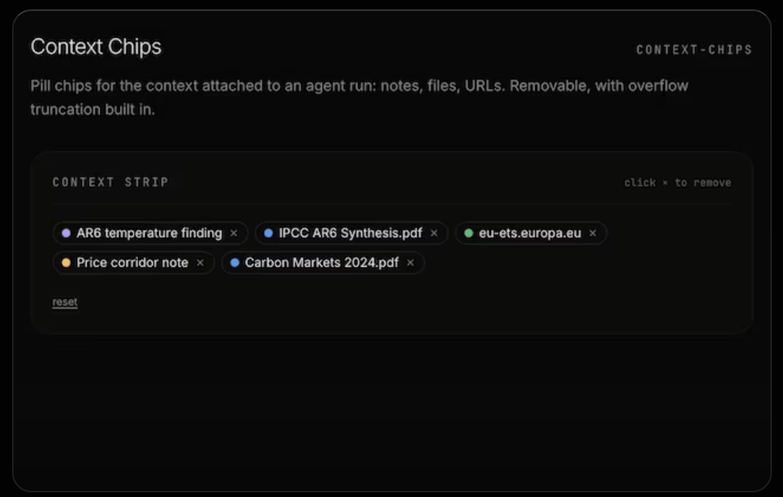
Tool Call Preview `tool-call-preview`



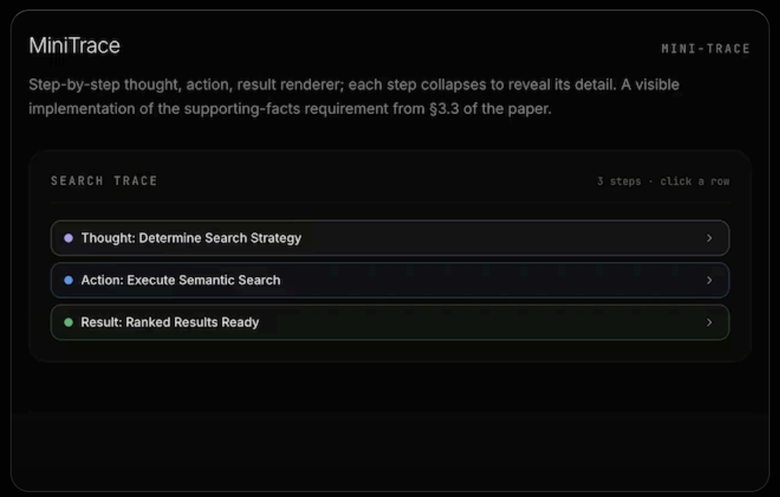
AI Generation Scale `ai-generation-*`



Shared Primitives `shared-primitives`



Context Chips `context-chips`



MiniTrace `mini-trace`

PRODUCT · IOS · SWIFTUI

BodyLog

A skin-tracking app for iPhone. You photograph a place on your body, say what it's about, and the app keeps the record. It never reads your skin, scores it, or tells you what to do — and nothing leaves the phone.

In simple terms

An iPhone app for tracking a visible skin or body condition between doctor visits: acne, eczema, psoriasis, cysts, bruising, physio progress.

It exists because of one question. If you have a chronic condition, the most useful thing your doctor asks is the hardest thing to answer: is it better or worse than last time? Six weeks have gone by, the flare that worried you has faded, and you are reconstructing it from memory in a ten-minute appointment.

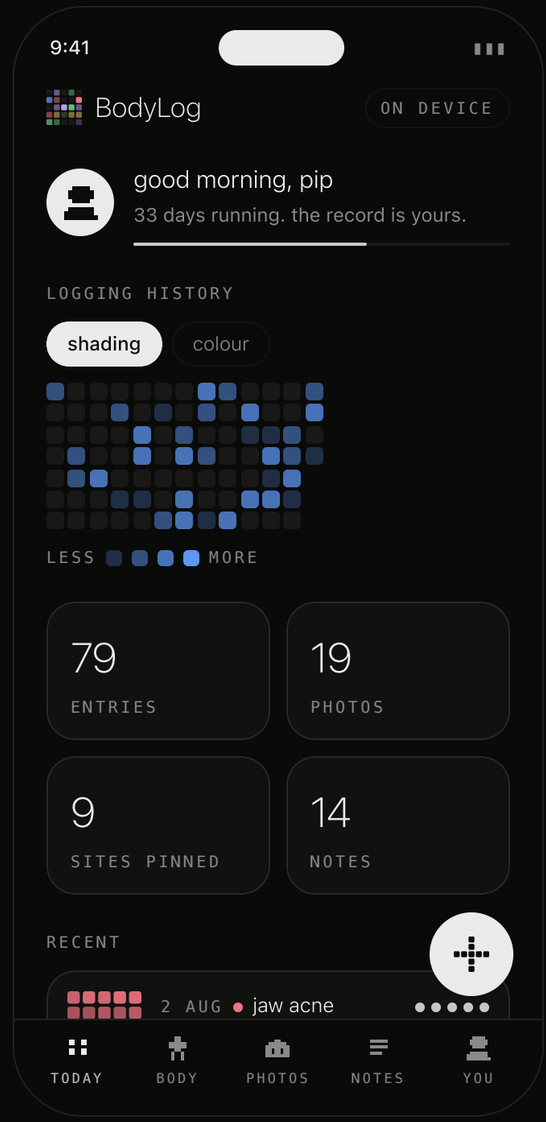
You photograph the thing when you notice it, tag where on the body it is, say how it feels, and move on. The whole job of the app is making that thirty-second habit sustainable and handing you the history when it matters.

It never examines your photos, never scores your skin, and never tells you what to do. Nothing leaves the phone: there is no network layer in the app. Not disabled, absent.

The impact. It turns an unanswerable appointment question into a record you can show someone, while holding the line that software with no medical training has no business grading a symptom.

$3/month or $25/year at launch. Native SwiftUI + SwiftData, iOS 17+, zero external dependencies — no image assets; every glyph, badge and figure is a character grid drawn at runtime.

THE APP, RENDERED LIVE



PERSONAL TOOL · MACOS · MIT

Blockpad

A macOS sketchpad that opens on a hotkey and hands drawings to whatever coding agent you're in. You draw where the boxes go, press copy, and paste. The agent gets the layout as exact structure, not a paragraph and not a screenshot.

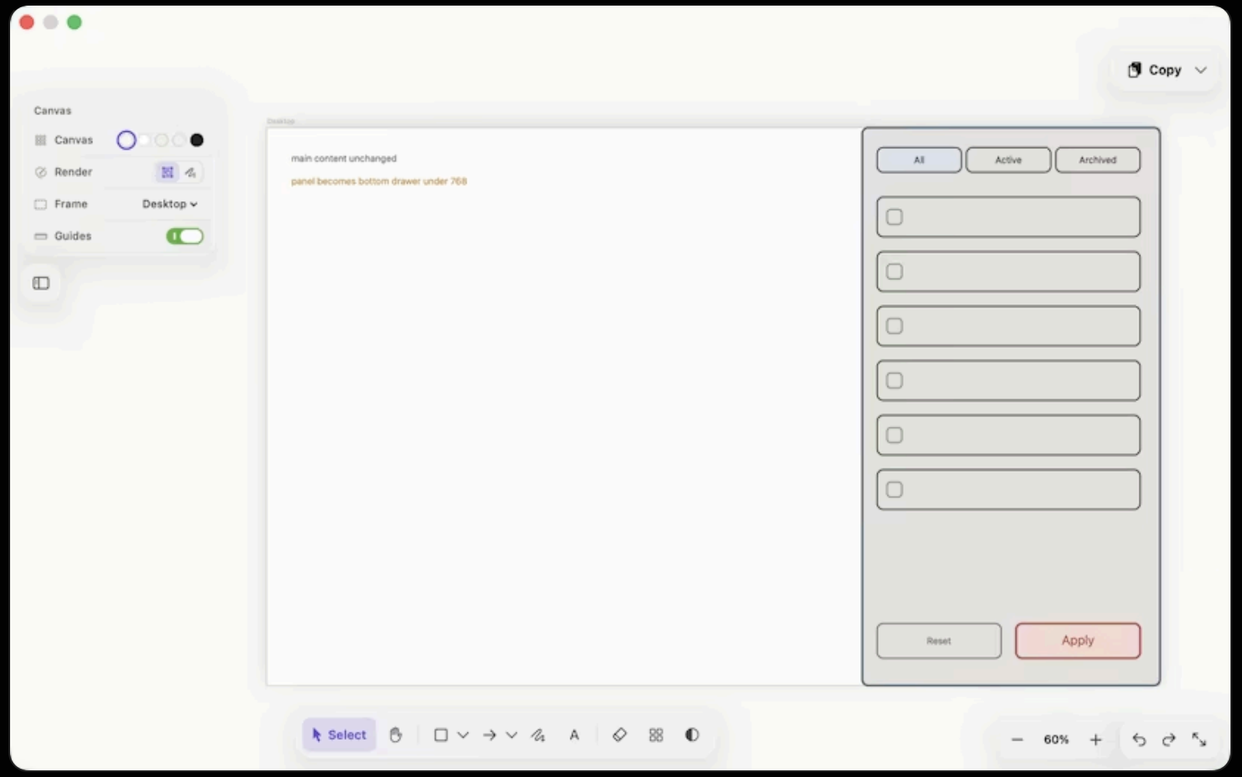
In simple terms

A sketchpad that opens over your code editor on a keyboard shortcut. You draw where the boxes go, press copy, and paste into a chat with a coding agent.

The problem it solves is narrow and expensive. You are in a repo and you need to say "filters go in a right-hand panel, tabs across the top, reset and apply in the footer." You type it. The agent builds something reasonable and wrong. You correct it. Three rounds later you have spent real tokens, real minutes and real attention reading implementations you are about to throw away. The cost is not the message, it is the rounds.

What gets pasted is the layout as exact structure: coordinates, sizes, colours, nesting. Not a paragraph, and not a screenshot the model has to interpret.

The impact. It removes the correction rounds, which is where the time and the tokens actually go. Free and MIT licensed, written in Swift with a single dependency, because it started as a tool for me.



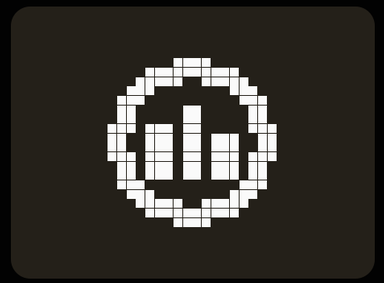
One window, one canvas, one Copy button. It opens over whatever you are already in, and the dock sits along the bottom so the top edge of the drawing stays clear.

Free and MIT licensed. Swift 6, SwiftUI, AppKit, macOS 14+, one dependency. Built for myself, open because there is no reason for it not to be.

`github.com/akaieuan/blockpad`

ALSO SHIPPED

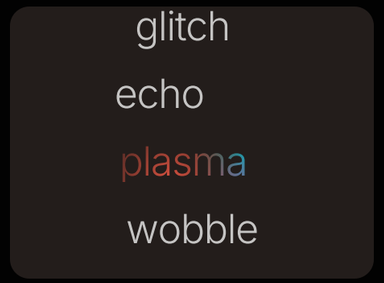
Twelve more, each with its own write-up at akabuild.dev/demo.



EVAL Kit

Open source

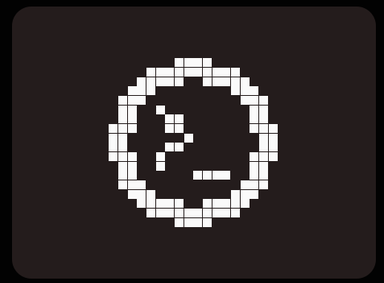
Agent eval framework where humans score, not LLMs. LLM-as-judge is only an opt-in pre-fill, flagged on every score. YAML suites, local dashboard, CLI, and five dimensions LLM judges miss.



Trickle UI Kit

Open source

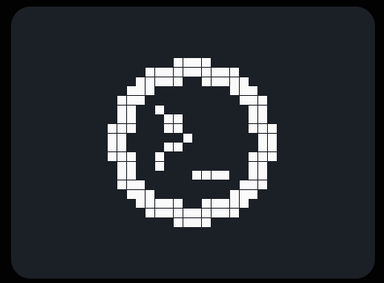
47 pure-CSS text-animation primitives for React. Zero runtime, SSR-safe, copy-paste install via the shadcn registry.



Collapse

Open source

Skills and MCP tools from your lessons. Three pluggable ingestors (MDX, Jupyter/MyST, and a one-file pattern for anything else) feed a typed pipeline with local atomic writes to ~/.claude/skills/. v0.2.



Hologram

Open source

Live observability and a read-only MCP surface for Blender to glTF pipelines. Watch your AI agent work on your game assets in real time.

akaVST

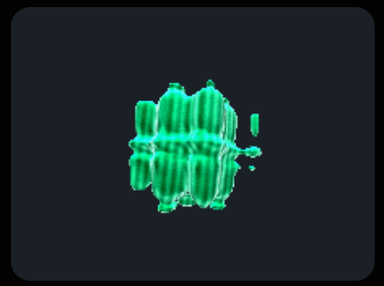
Instruments

Three JUCE instruments, built one at a time and documented as they go: an acid voice with a 64-step sequencer (v0.4.0), four lo-fi layers on one voice pool (v1.0.0), and a sampler that resamples itself (v0.1.0).

akaCOVART

Open source

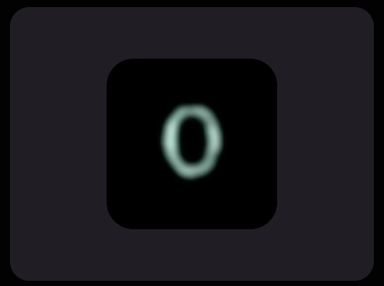
A generative album-art engine. Shape it, sync the motion to your track, and export the cover. Every cover is reproducible from an engine, a seed, and a few parameters.



Three.js Examples

Live demo

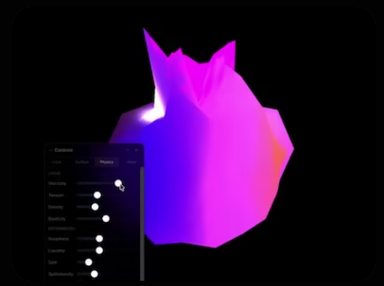
The liquid shader orb from the old landing hero, running live: hand-written GLSL over Three.js, and the rules that let WebGL sit on a portfolio page.



Null Browser

Open source

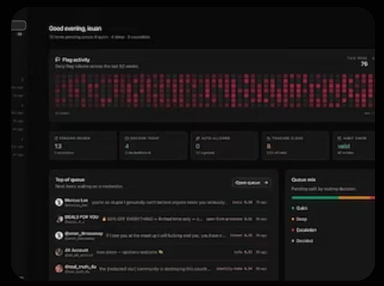
Privacy-first Tauri browser: zero telemetry, no AI, notes as markdown on disk, every connection visible and blockable. MPL 2.0.



Visualizer Eden

Audio tool

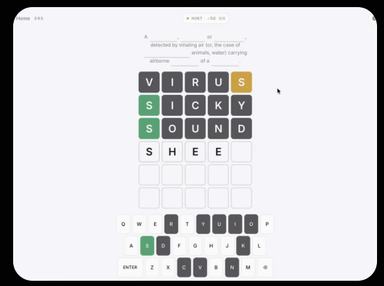
Browser-based 3D audio visualizer with reactive mesh deformation, custom GLSL shaders, and material presets.



Inertial - Content Moderation Tool

Open source

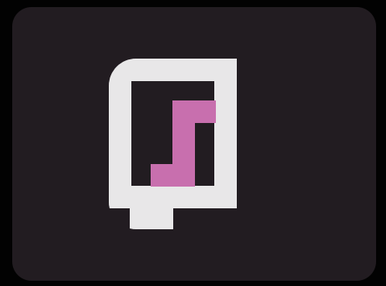
Reference architecture for auditable AI content review: typed signals, YAML policy, hash-chained audit log, reviewer dashboard, eval harness.



Wordle remake: Wrdef (Wordle + definition)

Game

A five-letter guessing game powered by definitions, bonus rounds, and a locally saved dictionary.



Music Analysis Chat

Interactive demo

Music analytics assistant with roster dashboards, creator discovery, and rich chat blocks.

WRITING

Built to Learn, Not to Help

akabuild.dev/writing/built-to-learn-not-to-help

ARCHIVE · UBIK

Benchmarks say AI passed the experts. One prompt through Liner and Ubik Studio, side by side, says otherwise.

The Pursuit of Parsimony [Pt.1]

akabuild.dev/writing/the-pursuit-of-parsimony

ESSAY · SCIENCE

When science meets surprise it contorts and survives through faith in ambiguity.

Digital Gentrification

akabuild.dev/writing/digital-gentrification

ESSAY · TECH

Notes on transformative information technology, from a Y2K kid.

Amplifying Human Intelligence

akabuild.dev/writing/amplifying-human-intelligence

ARCHIVE · UBIK

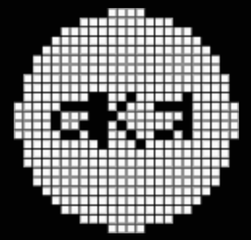
The design philosophy behind Ubik: build AI that amplifies human intelligence rather than outsourcing cognition.

The Ubik Portals Bible

akabuild.dev/writing/ubik-portals-bible

ARCHIVE · UBIK

The classroom spec: eight assignment types, a five-point AI assistance scale, and the 27 tools it gates.



Everything on this site is the same habit at different sizes: go and find the problem, build something you can actually operate, and keep the person using it in charge of what happens next.

REACH ME

akabuild.dev

ieuan@ubik.studio

github.com/akaieuan

The full index is at akabuild.dev/demo.